



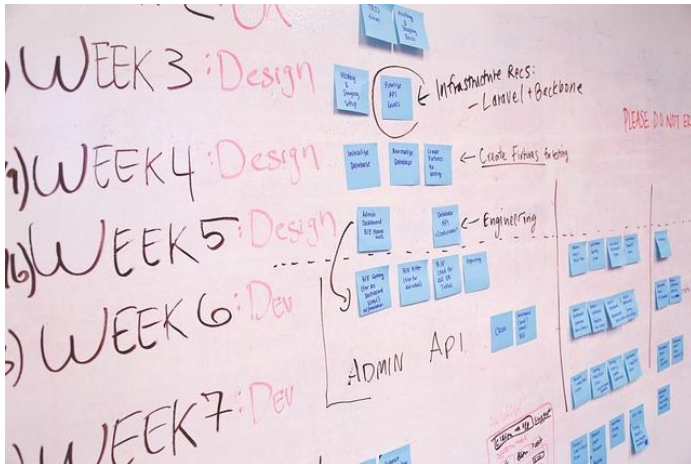
Unidad Didáctica

Metodologías de Desarrollo de Software, Unidad V

Contenido

Índice de Contenido	2
Introducción	3
Desarrollo de Contenidos	5
Metodologías SDL	5
Metodologías Ágiles.....	8
¿Qué es desarrollo ágil?	8
Programación Extrema (Extreme Programming XP).....	12
SCRUM.....	16
Metodologías orientadas a Objetos	21
Bibliografía y Webgrafía	22
Glosario	22

Introducción



Bienvenidos a la Unidad V de nuestro curso sobre Ingeniería del Software. En esta etapa, nos sumergiremos en el fascinante mundo de las Metodologías de Desarrollo de Software, explorando conceptos clave que van desde los fundamentos hasta

las prácticas más avanzadas en este campo tan dinámico y esencial para la industria de la tecnología.

Nuestros objetivos principales para esta unidad son múltiples y fundamentales para cualquier profesional en el ámbito de la Ingeniería del Software. En primer lugar, buscamos comprender los fundamentos de las metodologías de diseño, incluyendo SDL, la metodología ágil y la metodología orientada a objetos. Esto sentará las bases para un entendimiento más profundo de los procesos de desarrollo de software desde una perspectiva ingenieril.

Además, nos esforzaremos por entender las razones detrás de los métodos de desarrollo ágil de software, así como las diferencias fundamentales entre el desarrollo ágil y el dirigido por un plan. Esto nos permitirá apreciar los enfoques flexibles y adaptativos que son cada vez más comunes en la industria del desarrollo de software y su importancia en la ingeniería del software.

Una parte crucial de esta unidad será conocer las prácticas clave en la Programación Extrema (Extreme Programming) y cómo se relacionan con los principios generales de los métodos ágiles. Esta exploración nos permitirá entender cómo los equipos de

desarrollo pueden maximizar la eficiencia y la calidad a través de métodos ágiles específicos, aplicando principios ingenieriles.

Por último, nos sumergiremos en el enfoque de Scrum para la administración de proyectos ágiles, comprendiendo cómo esta metodología puede facilitar la gestión efectiva de proyectos en entornos dinámicos y cambiantes, lo cual es esencial desde la perspectiva de la ingeniería del software.

Para lograr estos objetivos, abordaremos el contenido de manera estructurada y detallada. Exploraremos las Metodologías SDL, comprendiendo su importancia y aplicaciones. Nos adentraremos en las Metodologías Ágiles, comenzando con una definición precisa y luego explorando en detalle la Programación Extrema (Extreme Programming) y el enfoque de Scrum. Finalmente, examinaremos las Metodologías Orientadas a Objetos, incluyendo OOSE, OMT y BOOCH, para comprender cómo se integran estas metodologías con los principios generales de la ingeniería del software.

¡Prepárense para un viaje emocionante hacia el mundo del desarrollo de software desde una perspectiva de ingeniería, donde exploraremos las mejores prácticas y las metodologías más innovadoras que impulsan la industria hoy en día!

Metodologías SDL

Las Metodologías SDL (Specification and Description Language) son un conjunto de herramientas y técnicas utilizadas en la ingeniería del software para la especificación, diseño y descripción de sistemas complejos. Estas metodologías proporcionan un marco estructurado para abordar el desarrollo de software, permitiendo a los ingenieros crear especificaciones claras y detalladas que facilitan la comprensión y el desarrollo de sistemas complejos.

El lenguaje SDL fue desarrollado originalmente para la especificación de sistemas de telecomunicaciones, pero su aplicabilidad se ha extendido a otros dominios, incluyendo sistemas embebidos y tiempo real. SDL es un lenguaje formal que permite describir el comportamiento de un sistema a través de diagramas y notaciones gráficas, lo que facilita la visualización y comprensión de las especificaciones.

Beneficios de SDL

Uno de los principales beneficios de utilizar SDL es la capacidad de crear modelos precisos y no ambiguos del sistema que se va a desarrollar. Estos modelos pueden ser utilizados para realizar simulaciones y verificaciones, permitiendo identificar y corregir errores en las etapas tempranas del desarrollo. Además, SDL facilita la comunicación entre los miembros del equipo de desarrollo y otros interesados, ya que proporciona una representación clara y estandarizada del sistema.

Componentes de SDL

SDL se basa en la teoría de autómatas y utiliza conceptos como estados, transiciones y eventos para describir el comportamiento del sistema. Los diagramas SDL están compuestos por varios componentes, incluyendo bloques, procesos, señales y

canales, que se utilizan para representar la estructura y el flujo de control del sistema. Cada proceso en un diagrama SDL puede ser considerado como un autómatas finito que cambia de estado en respuesta a eventos, y los canales se utilizan para representar la comunicación entre los procesos.

Ejemplos Prácticos de SDL

Ejemplo 1: Sistema de Control de Semáforos

Un ejemplo clásico de la aplicación de SDL es el diseño de un sistema de control de semáforos. En este sistema, los semáforos deben cambiar de estado (verde, amarillo, rojo) en respuesta a eventos como el paso del tiempo o la detección de vehículos.

1. Modelado de Estados y Transiciones: Los estados pueden incluir Verde, Amarillo, y Rojo. Las transiciones entre estos estados se desencadenan por eventos como `TimerExpired` (temporizador expirado) o `EmergencyVehicleDetected` (vehículo de emergencia detectado).
2. Uso de Diagramas: Se utiliza un diagrama SDL para representar los estados y las transiciones. Cada estado del semáforo se representa como un nodo, y las transiciones se representan como flechas entre los nodos, etiquetadas con los eventos correspondientes.
3. Comunicación entre Semáforos: En un sistema de semáforos con múltiples intersecciones, los semáforos deben comunicarse entre sí para coordinarse. Esto se puede modelar utilizando canales SDL para enviar señales entre procesos que representan cada semáforo.

Ejemplo 2: Sistema de Gestión de Llamadas en Telecomunicaciones

SDL es ampliamente utilizado en la industria de telecomunicaciones para modelar sistemas de gestión de llamadas. En este caso, un sistema debe manejar múltiples llamadas entrantes y salientes, asegurando que se establezcan y terminen correctamente.

1. Modelado de Procesos: Se crean procesos SDL para representar diferentes componentes del sistema, como el CallManager, UserInterface, y NetworkInterface.
2. Interacción entre Procesos: Los procesos se comunican a través de señales SDL. Por ejemplo, cuando un usuario inicia una llamada, el UserInterface envía una señal InitiateCall al CallManager. El CallManager procesa la señal, verifica la disponibilidad de recursos y, si es posible, envía una señal SetupCall al NetworkInterface para establecer la conexión.
3. Manejo de Errores: Si ocurre un error durante el establecimiento de la llamada, como la falta de recursos, el CallManager envía una señal de error de vuelta al UserInterface, que luego notifica al usuario del problema.

Enfoque Iterativo e Incremental

El uso de SDL en el desarrollo de software sigue un enfoque iterativo e incremental, lo que permite refinamientos y mejoras continuas en el modelo del sistema. Esto es particularmente útil en proyectos complejos donde los requisitos pueden cambiar durante el desarrollo. La capacidad de realizar simulaciones y verificaciones permite a los ingenieros detectar y corregir problemas de manera temprana, reduciendo el costo y el tiempo asociados con la corrección de errores en etapas posteriores.

En resumen, las Metodologías SDL son una herramienta poderosa en la ingeniería del software para la especificación, diseño y descripción de sistemas complejos. Su uso permite crear modelos precisos y no ambiguos, facilita la comunicación entre los miembros del equipo y otros interesados, y proporciona una base sólida para la simulación y verificación del sistema. Al adoptar SDL, los ingenieros pueden mejorar la calidad y eficiencia del desarrollo de software, asegurando que los sistemas cumplen con los requisitos y funcionan correctamente en su entorno operativo. Los ejemplos prácticos, como el sistema de control de semáforos y el sistema de gestión

de llamadas, ilustran cómo SDL puede aplicarse efectivamente en diferentes contextos para resolver problemas complejos y mejorar el desarrollo de software.

Metodologías Ágiles

¿Qué es desarrollo ágil?

El desarrollo ágil es un enfoque para la gestión de proyectos y el desarrollo de software que se centra en la entrega rápida y continua de valor a través de la colaboración iterativa y la capacidad de respuesta al cambio. Este método contrasta con los enfoques tradicionales de desarrollo de software, como el modelo en cascada, que siguen un proceso secuencial y rígido. En el desarrollo ágil, los equipos trabajan en ciclos cortos e iterativos llamados sprints, donde se planifican, desarrollan, prueban y revisan pequeños incrementos del producto.

Principios del Desarrollo Ágil

Los principios del desarrollo ágil se derivan del Manifiesto Ágil, un documento creado en 2001 por un grupo de desarrolladores de software que buscaban una mejor manera de gestionar los proyectos de software. El Manifiesto Ágil establece cuatro valores fundamentales y doce principios que guían la práctica ágil:

Valores Fundamentales

1. Individuos e interacciones sobre procesos y herramientas: La colaboración entre personas es más importante que seguir procedimientos estrictos.
2. Software funcionando sobre documentación extensiva: La entrega de software funcional es más valiosa que la creación de documentación detallada.
3. Colaboración con el cliente sobre negociación de contratos: Es preferible trabajar con los clientes para satisfacer sus necesidades que adherirse estrictamente a contratos predeterminados.

4. Respuesta ante el cambio sobre seguir un plan: La capacidad de adaptarse a los cambios es más importante que seguir un plan fijo.

Principios del Manifiesto Ágil

1. Satisfacer al cliente a través de la entrega temprana y continua de software valioso.
2. Aceptar los requisitos cambiantes, incluso en etapas tardías del desarrollo.
3. Entregar software funcional con frecuencia, en intervalos de pocas semanas a pocos meses.
4. La colaboración diaria entre desarrolladores y el negocio.
5. Construir proyectos alrededor de individuos motivados, brindándoles el entorno y el apoyo necesarios.
6. La comunicación cara a cara es la forma más eficiente y efectiva de transmitir información.
7. El software funcionando es la medida principal de progreso.
8. Los procesos ágiles promueven el desarrollo sostenible, con un ritmo constante e indefinido.
9. La atención continua a la excelencia técnica y al buen diseño mejora la agilidad.
10. La simplicidad, o el arte de maximizar la cantidad de trabajo no realizado, es esencial.
11. Las mejores arquitecturas, requisitos y diseños emergen de equipos autoorganizados.
12. A intervalos regulares, el equipo reflexiona sobre cómo ser más efectivo y ajusta su comportamiento en consecuencia.

Prácticas Ágiles

El desarrollo ágil incluye una serie de prácticas que ayudan a implementar estos principios y valores. Algunas de las más comunes incluyen:

- Scrum: Un marco ágil que divide el trabajo en sprints, facilita reuniones diarias (stand-ups) y revisiones regulares (retrospectivas) para mejorar continuamente.
- Kanban: Una metodología visual que utiliza tableros para gestionar el flujo de trabajo y mejorar la eficiencia.
- Programación Extrema (XP): Un conjunto de prácticas técnicas que incluyen desarrollo dirigido por pruebas (TDD), integración continua y programación en pareja.
- Lean Software Development: Un enfoque que busca minimizar el desperdicio y maximizar el valor entregado al cliente.

Beneficios del Desarrollo Ágil

- Flexibilidad y Adaptabilidad: Los equipos ágiles pueden responder rápidamente a los cambios en los requisitos y prioridades del cliente.
- Mejora Continua: Las retrospectivas periódicas permiten a los equipos identificar y abordar problemas, mejorando constantemente sus procesos y productos.
- Mayor Satisfacción del Cliente: La entrega frecuente de software funcional permite a los clientes ver y utilizar el producto en desarrollo, proporcionando retroalimentación valiosa.
- Riesgo Reducido: Los ciclos cortos de desarrollo permiten identificar y mitigar riesgos de manera temprana.
- Colaboración Mejorada: La comunicación constante entre todos los miembros del equipo, incluidos los clientes, facilita una mejor comprensión de los requisitos y expectativas.

Ejemplo Práctico de Desarrollo Ágil

Proyecto de Desarrollo de una Aplicación Móvil

1. Inicio del Proyecto:

- Reunión de Inicio: El equipo se reúne con el cliente para definir la visión del producto y las características iniciales.
- Backlog del Producto: Se crea un backlog del producto con todas las características deseadas, priorizadas por el valor para el cliente.

2. Planificación del Sprint:

- Selección de Tareas: El equipo selecciona las tareas del backlog que pueden completarse en un sprint de dos semanas.
- Definición de Hecho: Se acuerda una "definición de hecho" para asegurar que todas las tareas completadas cumplan con los criterios de calidad.

3. Desarrollo del Sprint:

- Reuniones Diarias: El equipo se reúne diariamente para discutir el progreso, los obstáculos y los próximos pasos.
- Desarrollo Iterativo: Los desarrolladores implementan las características, siguiendo prácticas como TDD e integración continua.

4. Revisión y Retroalimentación:

- Demostración del Sprint: Al final del sprint, el equipo presenta las características completadas al cliente para obtener retroalimentación.
- Retrospectiva del Sprint: El equipo reflexiona sobre lo que funcionó bien y lo que puede mejorar en el próximo sprint.

5. Adaptación y Mejora:

- Ajustes en el Backlog: Basado en la retroalimentación del cliente, el backlog se actualiza y reprioriza.
- Implementación de Mejoras: Las mejoras identificadas durante la retrospectiva se implementan en el próximo sprint.

El desarrollo ágil es un enfoque dinámico y colaborativo que permite a los equipos de software adaptarse rápidamente a los cambios y entregar valor continuo a los clientes. A través de ciclos iterativos y la implementación de prácticas ágiles, los equipos pueden mejorar constantemente sus procesos, reducir riesgos y aumentar la satisfacción del cliente. Al adoptar los principios y valores del Manifiesto Ágil, los equipos pueden crear software de alta calidad de manera eficiente y efectiva, respondiendo a las necesidades cambiantes del mercado y de los usuarios.

Programación Extrema (Extreme Programming XP)

La Programación Extrema (XP) es una metodología de desarrollo de software ágil que se centra en mejorar la calidad del software y la capacidad de respuesta a los requisitos cambiantes del cliente. Creada por Kent Beck en la década de 1990, XP promueve la colaboración estrecha entre los desarrolladores y los clientes, así como un enfoque iterativo e incremental para el desarrollo de software.

Principios y Valores de XP

XP se basa en cinco valores fundamentales que guían todas las prácticas y principios del equipo:

1. Comunicación: La comunicación abierta y frecuente entre todos los miembros del equipo, incluidos los clientes, es crucial para asegurar que todos estén alineados y que los problemas se aborden rápidamente.
2. Simplicidad: El diseño y el código deben ser lo más simples posible, evitando complejidades innecesarias y facilitando futuras modificaciones.
3. Retroalimentación: Obtener retroalimentación constante y oportuna de los clientes y del propio equipo ayuda a ajustar el rumbo del desarrollo y mejorar continuamente.
4. Coraje: Los miembros del equipo deben tener el valor de enfrentar los problemas, tomar decisiones difíciles y cambiar direcciones cuando sea necesario.

5. Respeto: Fomentar un ambiente de respeto mutuo donde cada miembro del equipo se sienta valorado y motivado.

Prácticas Clave de XP

XP implementa una serie de prácticas clave que ayudan a poner en acción sus principios y valores. Estas prácticas están diseñadas para mejorar la calidad del software y la eficiencia del equipo.

1. Desarrollo Dirigido por Pruebas (TDD):
 - Escribir pruebas automatizadas antes de desarrollar el código funcional.
 - Las pruebas aseguran que el código cumple con los requisitos y permite refactorizar sin temor a introducir errores.
2. Integración Continua:
 - Integrar y probar el código frecuentemente, al menos una vez al día.
 - Detectar y solucionar problemas de integración de manera temprana.
3. Refactorización:
 - Mejorar continuamente el diseño del código sin cambiar su funcionalidad.
 - Mantener el código limpio y manejable, facilitando futuras modificaciones.
4. Programación en Pareja (Pair Programming):
 - Dos desarrolladores trabajan juntos en una sola estación de trabajo.
 - Mejora la calidad del código y fomenta la colaboración y el intercambio de conocimientos.
5. Propiedad Colectiva del Código:
 - Todo el equipo es responsable del código base.
 - Cualquier miembro puede modificar cualquier parte del código, promoviendo la responsabilidad compartida.
6. Simplicidad en el Diseño:
 - Diseñar solo lo necesario para cumplir con los requisitos actuales.

- Evitar la sobreingeniería y centrarse en soluciones simples y efectivas.
7. Releases Frecuentes:
- Entregar versiones funcionales del software con frecuencia.
 - Permitir a los clientes ver y probar el software regularmente, proporcionando retroalimentación valiosa.
8. Metáfora del Sistema:
- Utilizar una metáfora simple para describir el sistema y guiar su desarrollo.
 - Facilita la comunicación y comprensión del diseño.
9. Planificación de Juegos (Planning Game):
- Clientes y desarrolladores colaboran para definir y priorizar historias de usuario.
 - Planificar iteraciones basadas en la importancia de las historias y la capacidad del equipo.
10. Semana de Trabajo Sostenible:
- Mantener un ritmo de trabajo constante y sostenible.
 - Evitar horas extras excesivas que puedan llevar al agotamiento del equipo.

Ejemplos Prácticos de XP

Ejemplo 1: Desarrollo de una Aplicación Web

1. Reunión de Planificación (Planning Game):
 - El equipo se reúne con el cliente para definir las historias de usuario para la próxima iteración.
 - Las historias se priorizan y se asignan puntos de estimación.
2. Desarrollo Dirigido por Pruebas (TDD):

- Antes de escribir cualquier código, los desarrolladores crean pruebas unitarias para cada historia de usuario.
 - Las pruebas fallan inicialmente, indicando que el código funcional aún no se ha implementado.
3. Programación en Pareja (Pair Programming):
- Dos desarrolladores trabajan juntos en cada historia, uno escribe el código mientras el otro revisa y sugiere mejoras.
 - Cambian de roles frecuentemente para mantener el dinamismo y la colaboración.
4. Integración Continua:
- El código se integra y se prueba automáticamente varias veces al día.
 - Los errores se detectan y corrigen rápidamente.
5. Refactorización:
- A medida que el código se desarrolla, se refactoriza regularmente para mejorar su estructura y eliminar duplicaciones.
 - Las pruebas automatizadas aseguran que la funcionalidad se mantiene intacta.

Ejemplo 2: Sistema de Control de Inventario

1. Simplicidad en el Diseño:
- El equipo se centra en implementar solo las características necesarias para la iteración actual.
 - Las decisiones de diseño complejas se posponen hasta que sean absolutamente necesarias.
2. Releases Frecuentes:
- Al final de cada iteración, se entrega una versión funcional del sistema al cliente.

- El cliente prueba el sistema y proporciona retroalimentación inmediata.

3. Propiedad Colectiva del Código:

- Todos los miembros del equipo revisan y mejoran el código continuamente.
- La responsabilidad compartida asegura que el código sea mantenible y de alta calidad.

La Programación Extrema (XP) es una metodología ágil que promueve la colaboración estrecha, la calidad del código y la capacidad de respuesta al cambio. A través de prácticas como TDD, integración continua y programación en pareja, XP ayuda a los equipos a entregar software funcional de alta calidad de manera eficiente y efectiva. Al enfocarse en valores como la comunicación, simplicidad, retroalimentación, coraje y respeto, XP crea un entorno de trabajo dinámico y colaborativo que mejora continuamente tanto los procesos como los productos.

SCRUM

Scrum es una metodología ágil que se utiliza para gestionar y controlar proyectos de desarrollo de software, aunque también puede aplicarse a otros tipos de proyectos. Es un marco de trabajo que facilita la colaboración entre equipos y promueve la entrega incremental de productos de alta calidad. Scrum se basa en un conjunto de roles, eventos y artefactos que ayudan a estructurar y gestionar el trabajo de manera eficiente.

Principios y Valores de Scrum

Scrum se fundamenta en los valores del Manifiesto Ágil y se enfoca en la transparencia, la inspección y la adaptación. Los cinco valores fundamentales de Scrum son:

4. Compromiso: Los miembros del equipo se comprometen a alcanzar los objetivos del sprint y a trabajar juntos para superarlos.
5. Coraje: El equipo debe tener el coraje de enfrentar desafíos y tomar decisiones difíciles.
6. Enfoque: Los equipos se concentran en el trabajo del sprint y en los objetivos del equipo.
7. Apertura: Los equipos y sus miembros son transparentes y honestos sobre su progreso y obstáculos.
8. Respeto: Los miembros del equipo se respetan mutuamente y valoran sus contribuciones.

Roles en Scrum

Scrum define tres roles principales que son esenciales para su funcionamiento:

1. Product Owner (Propietario del Producto):
 - Responsable de maximizar el valor del producto y del trabajo del equipo de desarrollo.
 - Define y prioriza el backlog del producto.
 - Actúa como el enlace entre el equipo de desarrollo y los interesados.
2. Scrum Master:
 - Facilita el proceso Scrum y asegura que se sigan las prácticas y principios de Scrum.
 - Elimina obstáculos que puedan impedir el progreso del equipo.
 - Promueve la comunicación y la colaboración dentro del equipo.
3. Development Team (Equipo de Desarrollo):
 - Compuesto por profesionales que trabajan en la entrega del incremento del producto.
 - Es autoorganizado y multifuncional, capaz de decidir cómo llevar a cabo su trabajo.
 - Colabora estrechamente para cumplir con los objetivos del sprint.

Eventos en Scrum

Scrum se organiza en torno a una serie de eventos que estructuran el trabajo y facilitan la inspección y adaptación:

1. Sprint:

- Un sprint es un ciclo de trabajo fijo de una a cuatro semanas durante el cual se crea un incremento del producto.
- Los sprints son de duración constante, lo que proporciona un ritmo regular para el equipo.

2. Sprint Planning (Planificación del Sprint):

- Una reunión al inicio del sprint donde se define qué se va a trabajar durante el sprint.
- El equipo de desarrollo selecciona los elementos del backlog del producto que se comprometen a completar.

3. Daily Scrum (Scrum Diario):

- Una reunión diaria de 15 minutos donde el equipo de desarrollo revisa el progreso hacia el objetivo del sprint.
- Los miembros del equipo comparten lo que hicieron ayer, lo que harán hoy y cualquier obstáculo que estén enfrentando.

4. Sprint Review (Revisión del Sprint):

- Una reunión al final del sprint donde se presenta el trabajo completado a los interesados.
- Se discute lo que se logró durante el sprint y se obtiene retroalimentación.

5. Sprint Retrospective (Retrospectiva del Sprint):

- Una reunión al final del sprint donde el equipo reflexiona sobre su desempeño.
- Identifica lo que funcionó bien, lo que no funcionó y cómo pueden mejorar en el próximo sprint.

Artefactos en Scrum

Scrum utiliza tres artefactos principales que proporcionan transparencia y oportunidades para la inspección y adaptación:

1. Product Backlog (Backlog del Producto):
 - Una lista priorizada de todo el trabajo que podría ser necesario en el producto.
 - Es administrada y priorizada por el Product Owner.
2. Sprint Backlog (Backlog del Sprint):
 - Una lista de los elementos del backlog del producto que el equipo de desarrollo se compromete a completar durante el sprint.
 - Incluye un plan detallado para entregar el incremento del producto.
3. Incremento:
 - La suma de todos los elementos del backlog del producto completados durante un sprint y los sprints anteriores.
 - Debe estar en un estado utilizable y cumplir con la "Definición de Hecho".

Beneficios de Scrum

4. Transparencia: Todos los miembros del equipo y los interesados tienen visibilidad del progreso del proyecto y del trabajo que se está realizando.
5. Flexibilidad y Adaptabilidad: Scrum permite responder rápidamente a los cambios en los requisitos y las prioridades del cliente.
6. Mejora Continua: Las retrospectivas periódicas ayudan al equipo a identificar y abordar problemas, mejorando continuamente sus procesos y productos.
7. Colaboración Mejorada: La estructura de roles y eventos fomenta la comunicación y la colaboración dentro del equipo y con los interesados.
8. Entrega Incremental: La entrega regular de incrementos del producto permite a los clientes ver y utilizar el software en desarrollo, proporcionando retroalimentación valiosa.

Ejemplo Práctico de Scrum

Proyecto de Desarrollo de una Aplicación de Comercio Electrónico

1. Inicio del Proyecto: El Product Owner crea un backlog del producto con todas las características deseadas, como el catálogo de productos, el carrito de compras, y el sistema de pago.
2. Sprint Planning (Planificación del Sprint):
 - El equipo se reúne para seleccionar las características más prioritarias para el primer sprint, como la interfaz de usuario para el catálogo de productos y la funcionalidad de búsqueda.
 - El equipo define las tareas necesarias para completar estas características y las incluye en el backlog del sprint.
3. Desarrollo del Sprint:
 - Daily Scrum (Scrum Diario): Cada día, el equipo se reúne para revisar el progreso y ajustar el plan según sea necesario.
 - Trabajo en las Tareas: Los desarrolladores trabajan en las tareas del backlog del sprint, implementando las características y realizando pruebas.
4. Sprint Review (Revisión del Sprint):
 - Al final del sprint, el equipo presenta el catálogo de productos y la funcionalidad de búsqueda a los interesados.
 - Los interesados proporcionan retroalimentación y sugieren mejoras o cambios.
5. Sprint Retrospective (Retrospectiva del Sprint): El equipo reflexiona sobre el sprint, discutiendo lo que funcionó bien, lo que podría mejorar y cómo pueden aplicar estos aprendizajes en el próximo sprint.
6. Preparación para el Siguiente Sprint:
 - El Product Owner actualiza el backlog del producto basado en la retroalimentación recibida.

- El equipo planifica el próximo sprint, seleccionando las siguientes características prioritarias, como la funcionalidad del carrito de compras.

Scrum es un marco ágil que facilita la colaboración efectiva y la entrega incremental de productos de alta calidad. A través de sus roles definidos, eventos estructurados y artefactos claros, Scrum ayuda a los equipos a gestionar proyectos de manera eficiente y a adaptarse rápidamente a los cambios. Al enfocarse en la transparencia, la inspección y la adaptación, Scrum promueve un entorno de mejora continua que beneficia tanto a los equipos de desarrollo como a los clientes.

Metodologías orientadas a Objetos

Las metodologías orientadas a objetos son enfoques sistemáticos para el análisis, diseño y desarrollo de sistemas de software utilizando conceptos de la programación orientada a objetos. Estas metodologías se centran en representar el mundo real a través de objetos, que combinan datos y comportamiento. Los objetos interactúan entre sí para formar sistemas complejos.

OOSE

OOSE, o Ingeniería de Software Orientada a Objetos, es una metodología desarrollada por Ivar Jacobson en la década de 1990. OOSE fue una de las primeras metodologías en introducir el concepto de casos de uso como una técnica central para el análisis y diseño de sistemas orientados a objetos.

Principios y Conceptos Clave de OOSE

1. Casos de Uso:

- Descripciones de cómo los usuarios interactúan con el sistema para lograr un objetivo específico.

- Los casos de uso ayudan a capturar los requisitos funcionales del sistema de manera clara y comprensible.
2. Análisis Orientado a Objetos:
 - Identificación de objetos clave y sus relaciones en el dominio del problema.
 - Creación de diagramas de clases que representan la estructura estática del sistema.
 3. Diseño Orientado a Objetos:
 - Definición de la arquitectura del sistema y la interacción entre los objetos.
 - Uso de patrones de diseño para resolver problemas comunes de diseño.
 4. Implementación y Pruebas:
 - Transformación del diseño en código fuente utilizando un lenguaje de programación orientado a objetos.
 - Pruebas unitarias y de integración para asegurar la calidad del sistema.

Ejemplo Práctico de OOSE

1. Caso de Uso:
 - Desarrollar un sistema de reserva de vuelos donde los usuarios puedan buscar y reservar vuelos.
 - Casos de uso: "Buscar vuelo", "Reservar vuelo", "Cancelar reserva".
2. Análisis:
 - Identificar objetos: Usuario, Vuelo, Reserva.
 - Definir relaciones: Un Usuario puede tener varias Reservas; una Reserva está asociada a un Vuelo.
3. Diseño:
 - Crear diagramas de clases para representar la estructura del sistema.
 - Definir métodos y atributos de cada clase.
4. Implementación y Pruebas:
 - Escribir el código para las clases y métodos definidos.

- Realizar pruebas para asegurar que las funcionalidades cumplen con los requisitos de los casos de uso.

OMT

OMT, o Técnica de Modelado de Objetos, es una metodología desarrollada por James Rumbaugh y sus colegas en la década de 1980. OMT proporciona un enfoque para el análisis y diseño de sistemas orientados a objetos utilizando modelos gráficos.

Principios y Conceptos Clave de OMT

1. Modelado de Objetos:

- Creación de un modelo de objetos que representa la estructura estática del sistema.
- Diagramas de clases que muestran objetos, atributos, métodos y relaciones.

2. Modelado Dinámico:

- Representación del comportamiento dinámico del sistema mediante diagramas de estados y eventos.
- Diagramas de secuencia y de colaboración que muestran la interacción entre objetos.

3. Modelado Funcional:

- Descripción de la funcionalidad del sistema utilizando diagramas de flujo de datos.
- Representación de procesos y transformaciones de datos.

Ejemplo Práctico de OMT

1. Modelado de Objetos:

- Desarrollar un sistema de gestión de biblioteca.
- Identificar objetos: Libro, Usuario, Préstamo.

- Definir relaciones: Un Usuario puede tener varios Préstamos; un Préstamo está asociado a un Libro.
2. Modelado Dinámico:
 - Crear diagramas de estados para el ciclo de vida de un Préstamo.
 - Diagramas de secuencia para la interacción entre Usuario y Sistema al realizar un Préstamo.
 3. Modelado Funcional:
 - Diagramas de flujo de datos para representar el proceso de préstamo de libros.
 - Definir las transformaciones de datos desde la solicitud del usuario hasta la actualización del inventario de libros.

BOOCH

La metodología Booch, desarrollada por Grady Booch, es una de las metodologías orientadas a objetos más completas y detalladas. Booch proporciona un enfoque riguroso para el análisis y diseño de sistemas utilizando una combinación de diagramas y documentación textual.

Principios y Conceptos Clave de Booch

1. Modelado de Objetos:
 - Identificación de clases y objetos, sus atributos, métodos y relaciones.
 - Diagramas de clases y objetos que representan la estructura del sistema.
2. Modelado de Estados:
 - Representación del comportamiento de los objetos a través de diagramas de estados.
 - Definición de eventos y transiciones de estado.
3. Modelado de Interacciones:

- Diagramas de interacción que muestran cómo los objetos colaboran para realizar funciones del sistema.
 - Diagramas de secuencia y de colaboración.
4. Modelado de Implementación:
- Diagramas de componentes y despliegue que representan la arquitectura física del sistema.
 - Descripción de cómo se implementan y despliegan los componentes del sistema.

Ejemplo Práctico de Booch

1. Modelado de Objetos:
- Desarrollar un sistema de ventas en línea.
 - Identificar objetos: Producto, Cliente, Pedido, Carrito de Compras.
 - Definir relaciones: Un Cliente puede hacer varios Pedidos; un Pedido contiene varios Productos.
2. Modelado de Estados:
- Crear diagramas de estados para el ciclo de vida de un Pedido.
 - Definir eventos como "Pedido Realizado", "Pedido Enviado", "Pedido Entregado".
3. Modelado de Interacciones:
- Diagramas de secuencia para la interacción entre Cliente y Sistema al añadir productos al Carrito de Compras.
 - Diagramas de colaboración para la verificación y procesamiento de un Pedido.
4. Modelado de Implementación:
- Diagramas de componentes que representan la estructura del sistema de ventas.
 - Diagramas de despliegue que muestran cómo se distribuyen los componentes en los servidores y la infraestructura de red.

Las metodologías orientadas a objetos, como OOSE, OMT y Booch, proporcionan enfoques estructurados para el análisis, diseño e implementación de sistemas de software. Cada metodología tiene sus propias técnicas y diagramas para representar los aspectos estáticos y dinámicos del sistema. Al utilizar estas metodologías, los desarrolladores pueden crear sistemas robustos y mantenibles que reflejen de manera efectiva los requisitos del mundo real y faciliten la colaboración entre los miembros del equipo.

Bibliografía y Webgrafía

- Montilva, J. C., & Barrios, J. A. (2021). Ingeniería del Software: Un enfoque basado en procesos (Edición en Español). Universidad de Los Andes.
- Martel, A. (2014). Gestión práctica de proyectos con Scrum: Desarrollo de software ágil para el Scrum Master (Aprender a ser mejor gestor de proyectos) (3rd ed., Edición en Español). CreateSpace Independent Publishing Platform.
- Simões, G. S., & Vazquez, C. E. (2018). Ingeniería de Requisitos: Software Orientado al Negocio (Spanish Edition). Independently published.
- Pressman, R. S. (2010). Ingeniería de Software (Spanish Edition) (7th ed.). McGraw-Hill Interamericana Editores S.A. de C.V.
- Agile y Scrum: Descubra el poder de la gestión de proyectos Agile, Lean Thinking, el proceso Kanban y Scrum (Spanish Edition)– 19 Diciembre 2020
- Edición en Español de Robert McCarthy (Author)
- McCarthy, R. (2020). Agile y Scrum: Descubra el poder de la gestión de proyectos Agile, Lean Thinking, el proceso Kanban y Scrum (Spanish Edition). Independently published.

Glosario

- Booch: Metodología orientada a objetos desarrollada por Grady Booch, que proporciona un enfoque riguroso para el análisis y diseño de sistemas.
- Casos de Uso: Descripciones de cómo los usuarios interactúan con el sistema para lograr un objetivo específico, fundamentales en la metodología OOSE.
- Daily Scrum: Reunión diaria de 15 minutos en Scrum donde el equipo revisa el progreso y ajusta el plan según sea necesario.
- Desarrollo Ágil: Enfoque de desarrollo de software que enfatiza la entrega incremental, la colaboración con los clientes y la capacidad de respuesta a los cambios.
- Desarrollo Dirigido por Pruebas (TDD): Práctica en XP donde se escriben pruebas automatizadas antes de desarrollar el código funcional.
- Diagrama de Clases: Representación gráfica de las clases, atributos, métodos y relaciones en un sistema orientado a objetos.
- Diagrama de Componentes: Representación gráfica de los componentes modulares del software y sus interacciones.
- Diagrama de Estados: Representación del comportamiento dinámico de un objeto mediante estados y transiciones.
- Diagrama de Secuencia: Representación gráfica de las interacciones temporales entre objetos en un sistema.
- Equipo de Desarrollo: Grupo de profesionales en Scrum que trabajan en la entrega del incremento del producto. Es autoorganizado y multifuncional.
- Extreme Programming (XP): Metodología ágil que se centra en la calidad del software y la capacidad de respuesta a los requisitos cambiantes del cliente.
- Ingeniería de Software Orientada a Objetos (OOSE): Metodología desarrollada por Ivar Jacobson que utiliza casos de uso para el análisis y diseño de sistemas orientados a objetos.

- Integración Continua: Práctica en XP de integrar y probar el código frecuentemente para detectar y solucionar problemas de integración tempranamente.
- Metodología Orientada a Objetos: Enfoque sistemático para el análisis, diseño y desarrollo de sistemas utilizando conceptos de programación orientada a objetos.
- Modelado Dinámico: Técnica en OMT para representar el comportamiento dinámico del sistema mediante diagramas de estados y eventos.
- Modelado de Objetos: Técnica en OMT para crear un modelo de objetos que represente la estructura estática del sistema.
- Modelado Funcional: Técnica en OMT para describir la funcionalidad del sistema utilizando diagramas de flujo de datos.
- Object Modeling Technique (OMT): Técnica de modelado de objetos desarrollada por James Rumbaugh que utiliza modelos gráficos para el análisis y diseño de sistemas.
- Pair Programming: Práctica en XP donde dos desarrolladores trabajan juntos en una sola estación de trabajo para mejorar la calidad del código y fomentar la colaboración.
- Planificación del Sprint: Reunión al inicio de cada sprint en Scrum donde se define el trabajo a realizar durante el sprint.
- Product Owner: Persona en Scrum responsable de maximizar el valor del producto y del trabajo del equipo de desarrollo.
- Refactorización: Práctica de mejorar continuamente el diseño del código sin cambiar su funcionalidad, común en XP.
- Scrum: Metodología ágil que utiliza roles, eventos y artefactos para gestionar proyectos de manera eficiente y promover la entrega incremental de productos de alta calidad.
- Sprint: Ciclo de trabajo fijo de una a cuatro semanas en Scrum durante el cual se crea un incremento del producto.

- Sprint Retrospective: Reunión al final de un sprint en Scrum donde el equipo reflexiona sobre su desempeño y busca mejorar continuamente.
- Transiciones: Cambios de estado en un diagrama de estados, representando el comportamiento dinámico de un objeto.